PROJECT REPORT

Project Title: PROCESS SYNCHRONIZATION SIMULATOR

Session: 2023-2027

(Reg. No. 2023-UOK-03680, 2023-UOK-03688, 2023-UOK-03691)

Submitted to: Mr. Mehtab Mushtaq

Course: OPERATING SYSTEM

Submitted By:

Muhammad Uzair, Syed Saqlain Haider, Ausaf Ahmad

Date of Submission: 13th Feburary,2026

DEPARTMENT OF CS & IT

FACULTY OF COMPUTING & ENGINEERING

UNIVERSITY OF KOTLI AZAD JAMMU & KASHMIR

PROCESS SYNCHRONIZATION SIMULATOR

(Reg. No. 2023-UOK-03680, 2023-UOK-03688, 2023-UOK-03691)

Submitted to: Mr. Mehtab Mushtaq

Course: OPERATING SYSTEM

Session: 2023-2027

Submitted By:

Muhammad Uzair, Syed Saqlain Haider, Ausaf Ahmad

Date of Submission: 13th Feburary,2026

DEPARTMENT OF CS & IT

FACULTY OF COMPUTING & ENGINEERING

UNIVERSITY OF KOTLI AZAD JAMMU & KASHMIR

Table of Contents

1.Introduction 5
2.Requirement Specifications 5
2.1 Functional Requirements 5
Process Management 5
Critical Section Handling 5
Synchronization Mechanisms 5
Control 5
Monitoring & Display 5
2.2 Non-Functional Requirements 6
3.Software Architecture Diagram 6
3.1 Architecture of UI and Simulation Engine 6
3.2 Connection of Simulation Engine to Managers 7
4.Component Architectural Diagram 8

Table of Figures

Figure 1.1……..……....5

Figure 1.2……..……..……6

Figure 1.3……..……7

1.Introduction

This is Version II of the project documentation for the Process Synchronization System Simulator. This version is intended to build and describe of software structure in the system. It covers major parts of the simulator, their connections and data flow which any user is supposed to know about system architecture before getting its hands dirty in implementation.

2.Requirement Specifications

2.1 Functional Requirements

The system must provide the following functionalities:

Process Management

- Allow the user to create multiple processes
- Assign each process an ID and execution time
- Simulate concurrent execution of processes

Critical Section Handling

- Define a shared resource accessed by multiple processes
- Clearly mark entry section, critical section, and exit section
- Ensure only one process enters the critical section at a time

Synchronization Mechanisms

- Implement synchronization techniques such as:
    - Mutex Locks
    - Semaphores (Binary and Counting)
- Allow the user to choose which mechanism to apply

Control

- Start, stop, reset, and step through the simulation
- Show process states (Running, Waiting, Blocked)

Monitoring & Display

- Show which process currently holds the lock/semaphore
- Display waiting queue of blocked processes
- Indicate deadlock or starvation if conditions arise (optional)

2.2 Non-Functional Requirements

The system must satisfy the following quality constraints:

- User interface should be simple and understandable
- System should respond in real-time during simulation
- Must run on Windows platforms
- Should support at least 10–15 concurrent processes
- Code should be modular and well-documented
- The simulator should not crash due to invalid input

3.Software Architecture Diagram

The component diagram of the Process Synchronization System illustrates the high-level architecture of the system and how its major modules interact. It shows the system as a set of components—such as the User Interface, Simulation Engine, Process Manager, Resource Manager, Synchronization Manager, and Deadlock Detection Module—each performing specific roles. The diagram also highlights how these components communicate through Required and Provided interfaces, using half-circles and dotted lines to indicate service requests and data flow. This visual representation helps in understanding how user commands are processed, how system resources are allocated, and how synchronization is maintained to ensure smooth execution of concurrent processes.

Furthermore, the diagram clearly shows the flow of process and resource data, illustrating which modules read from or write to shared data stores. The Deadlock Detection Module monitors resource allocation and process interactions to prevent system deadlocks. Overall, this component diagram provides a concise view of the system's internal structure, dependencies, and interactions, making it easier to analyze, maintain, and explain the architecture of the Process Synchronization System.

3.1 Architecture of UI and Simulation Engine

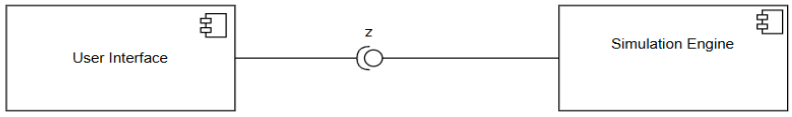

Figure 1.1

Explanation:

The connection between the User Interface and the Simulation Engine represents the first step in the system workflow. Here, the User Interface acts as a Required interface, while the Simulation Engine acts as a Provided interface. When a user provides input, such as executing a process or requesting memory allocation, this input is sent to the Simulation Engine. The engine processes the input and sends the execution results back to the User Interface so the user can see the output or system status. This connection is essential because the entire simulation depends on the input received from the user.

In the diagram, this interface is shown with a dotted line and a half-circle to indicate that the User Interface is requesting a service, and the Simulation Engine is providing it. This clearly shows the direction of service flow and maintains standard UML notation.

3.2 Connection of Simulation Engine to Managers

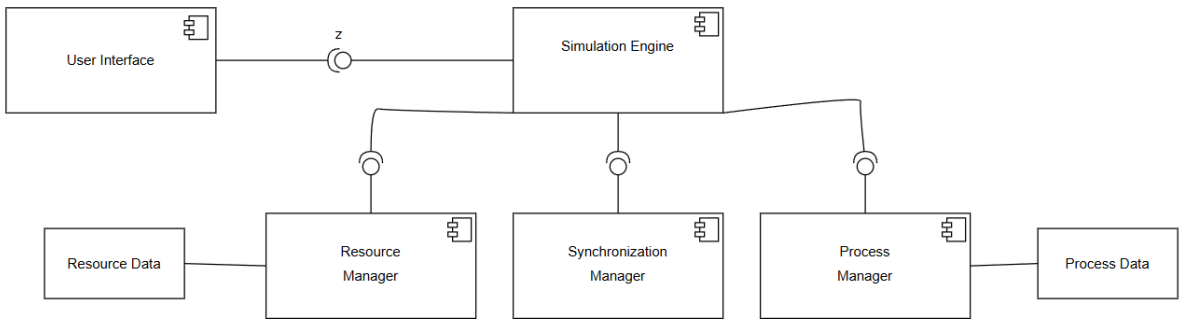

Figure 1.2

Explanation:

The Simulation Engine handles three key responsibilities: managing processes, allocating resources, and coordinating synchronization. These three components—Process Manager, Resource Manager, and Synchronization Manager—connect to the Simulation Engine through its Required interfaces. The Simulation Engine sends requests for process services, resource services, and synchronization services to these modules. These connections ensure that system operations are coordinated efficiently and help prevent deadlocks during execution.

In the diagram, these connections are represented with half-circles (Required interfaces) and dotted lines, which clearly indicate that the Simulation Engine requests services and the other modules provide the requested operations. This structure centralizes system logic and makes the workflow easy to follow.

4.Component Architectural Diagram

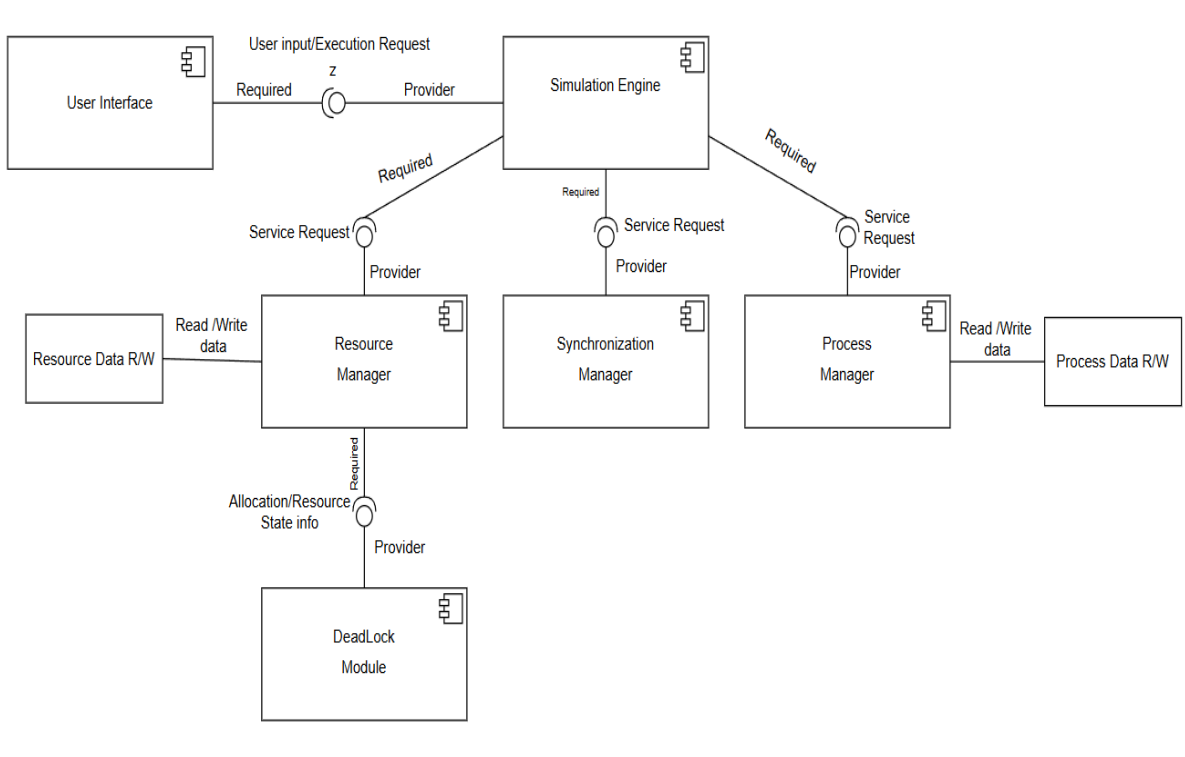

Figure 1.3

Explanation:

The complete component diagram shows the relationships and interactions among all system modules: User Interface, Simulation Engine, Process Manager, Synchronization Manager, Resource Manager, Process Data, Resource Data, and the Deadlock Detection Module. Required and Provided interfaces are clearly indicated using half-circles and dotted lines to represent service requests and data flow. The diagram demonstrates how user input flows through the system, how internal modules interact, and how output is generated.

It also highlights data access and service coordination. Process Data and Resource Data are clearly shown as read/write resources, while the Deadlock Detection Module receives allocation and resource state information.